

Error Handling

Write Robust, Production-Ready JavaScript

■■ ERROR HANDLING

■ LEARNING OUTCOME

By the end of this module you will handle all error types, create custom Error classes, implement global handlers, log errors professionally, and build resilient applications that fail gracefully.

■ Skills You Will Gain

- Use try/catch/finally for sync and async errors
- Create custom Error classes with meaningful messages
- Handle TypeError, RangeError, NetworkError
- Set up global error handlers
- Log errors with context for debugging
- Apply defensive programming patterns

■ Unhandled errors crash apps and frustrate users. Professional JavaScript handles every failure gracefully -- logging it, showing a user-friendly message, and recovering when possible.

SECTION 1

Error Types and try/catch/finally

	When It Occurs	Example
	Base error -- use for app errors	throw new Error('Something failed')
	Wrong type or null/undefined access	null.property or undefined()
	Value outside allowed range	new Array(-1) or stack overflow
	Access undefined variable	console.log(undeclaredVar)
	Invalid JS syntax	JSON.parse('{invalid}')

```
// try / catch / finally async function fetchWithCleanup(url) { showSpinner(); try { const res =
await fetch(url); if (!res.ok) throw new Error(`HTTP ${res.status}: ${res.statusText}`); return
await res.json(); } catch (error) { if (error instanceof TypeError) return showOfflineMsg();
logError(error, { url }); throw error; // re-throw unexpected errors } finally { hideSpinner(); //
ALWAYS runs } } // Catch specific types try { const data = JSON.parse(jsonString); if (typeof data
!== 'object') throw new TypeError('Expected object'); return data; } catch (e) { if (e instanceof
SyntaxError) { console.error('Invalid JSON'); return null; } throw e; // re-throw unknown errors }
```

SECTION 2

Custom Error Classes & Global Handlers

```
// Custom error classes class AppError extends Error { constructor(message, code, statusCode=500) {
super(message); this.name = this.constructor.name; this.code = code; this.statusCode = statusCode;
this.timestamp = new Date().toISOString(); } } class ValidationError extends AppError {
constructor(field, msg) { super(`Validation failed for '${field}': ${msg}`, 'VALIDATION_ERROR',
422); this.field = field; } } class NotFoundError extends AppError { constructor(resource, id) {
super(`${resource} '${id}' not found`, 'NOT_FOUND', 404); } } // Using custom errors function
getUser(id) { if (!id) throw new ValidationError('id', 'must not be empty'); const user =
users.find(u => u.id === id); if (!user) throw new NotFoundError('User', id); return user; } try {
const u = getUser(''); } catch (e) { if (e instanceof ValidationError) showFieldError(e.field,
e.message); else if (e instanceof NotFoundError) show404(); else throw e; } // GLOBAL HANDLERS
window.onerror = (message, source, line, col, error) => { logToServer({ message, source, line,
stack: error?.stack }); return true; }; window.addEventListener('unhandledrejection', (event) => {
logToServer({ message: event.reason?.message, stack: event.reason?.stack });
event.preventDefault(); });
```

■ PRACTICE Q & A

Q1. What is the difference between throw and return in error handling?

A: return exits the function with a value. throw terminates execution, propagates error up the call stack, and transfers control to nearest catch block.

Q2. When should you re-throw an error?

A: Re-throw when: the error is unexpected, you've logged it but caller needs to know, or you can't recover at this level. Always re-throw errors you don't know how to handle.

Q3. What does finally do?

A: Runs ALWAYS -- whether try succeeds, catch handles error, or code throws again inside catch. Use for cleanup: hide spinners, release locks, close connections.

Q4. Why create custom Error classes?

A: Custom errors carry structured metadata (statusCode, field, code), enable instanceof checks, have meaningful .name for logging, and create clear contracts between layers.

Q5. What is the difference between window.onerror and unhandledrejection?

A: onerror catches synchronous uncaught exceptions. unhandledrejection catches unhandled Promise rejections. Both needed for full coverage.

■■ Error Handling Cheat Sheet	
<code>try{}catch(e){}finally{}</code>	Full error handling block
<code>throw new Error('msg')</code>	Throw -- propagates up
<code>throw new TypeError('msg')</code>	Specific error type
<code>e instanceof TypeError</code>	Check error type
<code>e.message / e.stack</code>	Error info and stack trace
<code>class AppErr extends Error{}</code>	Custom error class
<code>finally { hideSpinner(); }</code>	Always runs -- cleanup
<code>throw error; // re-throw</code>	Pass to higher handler
<code>window.onerror = fn</code>	Global sync error handler
<code>unhandledrejection event</code>	Global async error handler
<code>navigator.sendBeacon(url,d)</code>	Log on page unload
<code>Error.captureStackTrace</code>	V8 stack trace capture